

Linear Search

Linear search checks elements one by one until it finds the target or reaches the end.

How It Works (Natural Explanation)

Imagine scanning a list from left to right with no shortcuts: - Compare the first element with the target. - If it does not match, move to the next element. - Continue until you either find the target or run out of elements.

Linear search does not need the array to be sorted. It is a universal fallback search method.

Formal View

Given an array $A[0..n-1]$ and a value x , linear search checks whether there exists an index i such that:

$$A[i] = x.$$

It evaluates this condition for indices in order $0, 1, 2, \dots, n-1$ and stops as soon as a match is found.

Why The Time Complexity Is What It Is

- Best case: $O(1)$
 - The target is at the first position, so only one comparison is needed.
- Worst case: $O(n)$
 - The target is at the end or not present, so all n elements are checked.
- Average case: $O(n)$
 - On average, a linear number of checks is required.

Reasoning: the number of comparisons is directly proportional to how far into the array the target is. In the worst and average cases, this grows linearly with n .

When To Use It

- Data is unsorted.
- Input is small, and simplicity is preferred.

- You need a quick baseline algorithm.
-

Strengths And Limitations

- Strengths:
 - Very simple to implement.
 - Works on any list structure (array, linked list, stream-like scan).
 - Limitations:
 - Does not scale well for large datasets compared to logarithmic search methods.
-

Related

- [\[\[Binary Search\]\]](#)
- [\[\[Searching Algorithms\]\]](#)
- [\[\[Lectures/Week 2\]\]](#)